OUTAGE DISSECTIONS • CONSENSUS PROTOCOLS • SPLIT-BRAIN FAILURE

Dissecting GitHub Outage: ZooKeeper Multiple Leaders Phenomenon

Curated from Arpit Bhayani's Architecture Dissection Series • Core Focus: Zab Consensus & Split-Brain

EXECUTIVE ARCHITECTURAL ABSTRACT

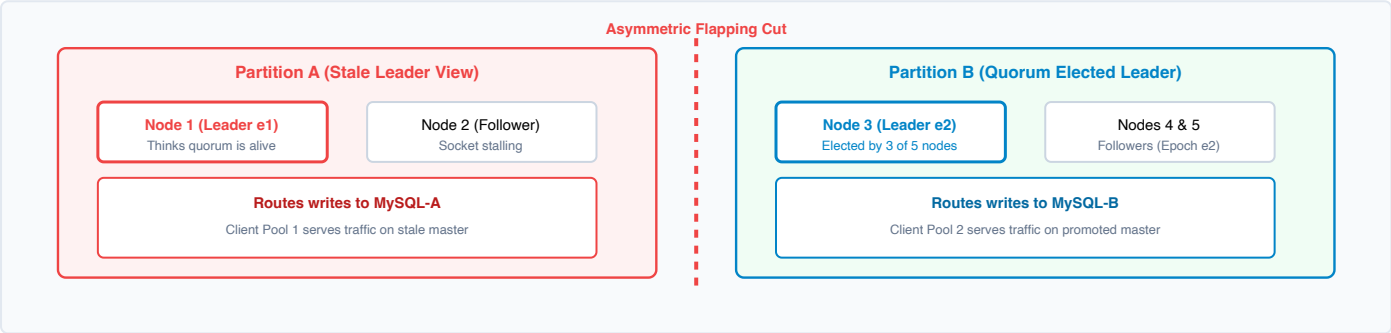
Distributed coordination systems (ZooKeeper, etcd) exist fundamentally to maintain a single, uncompromised source of truth. In this critical GitHub infrastructure outage, an **asymmetric network partition** coupled with lease expiration timing delays caused a catastrophic breakdown of Zab consensus: **two distinct ZooKeeper nodes simultaneously operated as cluster leaders**. Stale routing assignments split production traffic across two un-synchronized MySQL masters, creating profound write divergence.

1. THE STRATEGIC ROLE OF ZOOKEEPER IN DATABASE TOPOLOGIES

At GitHub, Apache ZooKeeper maintained authoritative metadata for database clustering and routing:

- **Active Master Registration:** The writable master for each database shard maintained an ephemeral node in ZooKeeper.
- **ProxySQL Routing Ingestion:** Proxies watched ZooKeeper trees to steer user SQL queries dynamically to the healthy primary.
- **Orchestrator Locks:** MySQL failover daemons used ZooKeeper distributed locks to serialize failover execution.

2. TOPOLOGY OF THE ASYMMETRIC NETWORK CUT



3. THE MECHANICS OF THE DUAL-LEADER SPLIT-BRAIN

How Zab consensus invariants were violated in production:

1. Asymmetric Heartbeat Loss

An intermittent switch failure dropped outbound packets from Leader 1 to Nodes 3, 4, 5. Nodes 3, 4, 5 observed consecutive missed pings and entered election mode. They held 3 out of 5 votes—a valid strict majority.

2. Delayed Demotion on Old Leader

Due to local garbage collection stalls and a generous socket read timeout, Leader 1 did not immediately detect that its followers had deserted. It continued responding to client reads, serving the old master pointer.

4. THE SOLUTION: FENCING TOKENS & MONOTONIC EPOCHS

A client must never trust a leader's lease blindly. Network latency and process pauses make lease boundaries fuzzy. The system must enforce **Fencing Tokens** at the storage layer:

The Martin Kleppmann Fencing Protocol

1. **Monotonic Token Issuance:** Every leader election in ZooKeeper increments the 32-bit epoch counter. When a client acquires a write lease, it receives a token: `Token = (Epoch, zxid)`.
2. **Storage Guard Invariant:** The target storage engine (MySQL / ProxySQL) records the maximum token it has ever observed: `MaxToken`.
3. **Fencing Verification on Every Write:**

```
def handle_write_query(query, client_fencing_token):
    if client_fencing_token < self.max_observed_token:
        # Client is acting on stale leadership! Drop immediately.
        raise StaleEpochException(f"Write rejected: Token {client_fencing_token} is stale")
    self.max_observed_token = client_fencing_token
    return self.execute_sql(query)
```

4. **Result:** Even if Leader 1 continues handing out leases, the storage engine rejects its writes because Leader 2 has already established a higher epoch.

5. ZOOKEEPER ENSEMBLE HARDENING ARCHITECTURE

To eliminate multiple-leader anomalies, GitHub hardened its consensus topologies:

- **Strict 3-Region Quorums:** Ensembles must be distributed across 3 Availability Zones (e.g., 2 nodes in Region A, 2 in Region B, 1 witness node in Region C). A severance between A and B can never create dual quorums.
- **Aggressive Leader Step-Down:** If a leader fails to receive acknowledgment from a majority of followers within `syncLimit * tickTime` (4 seconds), it immediately revokes its own leadership and drops all client connections.
- **Fail-Fast Client Disconnects:** When client libraries receive a `DISCONNECTED` event, they immediately throw read-only exceptions rather than continuing with cached routing tables.

6. PROTOCOL COMPARISON: ZOOKEEPER VS. ETCD VS. CONSUL

Consensus Engine	Consensus Algorithm	Split-Brain Resilience	Leader Step-Down Rule
Apache ZooKeeper	Zab (Atomic Broadcast)	High (Vulnerable if syncLimit is misconfigured)	Requires active heartbeats every tickTime * syncLimit
etcd (Kubernetes Core)	Raft	Strict (Leader lease read verification via Heartbeat quorums)	Leader checks quorum before answering linearizable reads
HashiCorp Consul	Raft	Strict (Serf Gossip + Raft Quorums)	Lease expiration enforces automatic step-down

7. PRODUCTION ENGINEERING TAKEAWAYS

1. **Never Rely on Client Leases Alone:** A client cannot know whether its lease has expired if it suffered a thread freeze, VM migration, or network pause. True split-brain prevention requires the storage layer to enforce monotonic fencing tokens.
2. **Two Datacenters Cannot Form a Fault-Tolerant Consensus Cluster:** Dividing a 5-node cluster as 3 and 2 across two sites guarantees that a network cut leaves one site dead and the other prone to split-brain. Consensus mathematically demands 3 independent failure domains.
3. **Linearizable Reads Require Quorum Affirmation:** A leader must verify it still holds quorum before serving critical routing decisions (e.g., in etcd's Read Index). Serving reads from memory without heartbeat validation invites split-brain disasters.